

FUTURE INTERNS

Cyber Security Internship 2026

API SECURITY RISK ANALYSIS REPORT

Professional API Security Assessment

Prepared By	Safal Kumar Singh
Domain	Cyber Security
Task	Task 03 – API Security Risk Analysis
Organization	Future Interns
Assessment Type	Read-Only Ethical API Testing
Report Version	1.0 Final
Date	May 2026

Table of Contents

1. Introduction
2. Objective
3. Scope & Ethics
4. APIs Tested
5. Tools & Technologies Used
6. Testing Methodology
7. API Security Findings & Analysis
 - 7.1 Missing Authentication Analysis
 - 7.2 Excessive Data Exposure Analysis
 - 7.3 Sequential Identifier / IDOR Analysis
 - 7.4 Security Header Analysis
 - 7.5 Rate Limiting Analysis
 - 7.6 ReqRes Public API Analysis
 - 7.7 ReqRes Security Header Analysis
8. OWASP API Security Top 10 Mapping
9. Positive Security Controls Identified
10. Remediation Recommendations
11. Conclusion
12. Disclaimer

1. Introduction

Modern applications rely heavily on APIs (Application Programming Interfaces) for communication between services, mobile applications, SaaS platforms, dashboards, and cloud systems.

If APIs are improperly secured, attackers may exploit them to:

- Access sensitive data
- Enumerate user information
- Abuse endpoints
- Bypass authentication
- Perform automated scraping
- Conduct reconnaissance activities

This project focuses on performing a professional API Security Risk Analysis using publicly accessible demo APIs in a safe and ethical environment.

The assessment involved:

- API endpoint testing
- Authentication analysis
- Response inspection
- HTTP header analysis
- Rate limiting inspection
- OWASP API Security mapping
- Security documentation creation

All testing was performed using ethical read-only methodologies.

2. Objective

The primary objectives of this project were:

- Analyze public APIs safely
- Identify common API security risks
- Inspect API authentication behavior
- Review exposed API response data
- Analyze security headers
- Identify excessive data exposure
- Understand API security best practices
- Map findings with OWASP API Security Top 10
- Create professional cybersecurity documentation

3. Scope & Ethics

Scope

The assessment was limited strictly to public educational/demo APIs.

APIs Used

JSONPlaceholder — <https://jsonplaceholder.typicode.com>

ReqRes — <https://reqres.in>

Ethical Guidelines Followed

- Read-only testing only
- No exploitation attempts
- No brute-force attacks
- No denial-of-service activities
- No unauthorized access attempts
- No bypass testing performed
- Public APIs only

4. APIs Tested

JSONPlaceholder API

Purpose: Public fake REST API for testing and learning.

Endpoints Tested: /users, /users/1, /posts, /comments

Authentication: Not Required | Protocol: HTTPS

ReqRes API

Purpose: Demo REST API platform for API testing.

Endpoints Tested: /api/users, /api/users/2

Authentication: Public Demo Access | Protocol: HTTPS

5. Tools & Technologies Used

Tool	Purpose
Postman	API Testing & Response Inspection
Browser DevTools	Header Inspection
VS Code	Script & Documentation Editing
Bash Scripting	Automated API Analysis
GitHub	Project Hosting
macOS Terminal	Script Execution
JSONPlaceholder	Public Demo API Testing
ReqRes	API Security Testing

6. Testing Methodology

Phase 1 — API Discovery

Public API endpoints were identified and documented.

Phase 2 — Authentication Testing

Endpoints were tested to determine whether authentication controls were implemented.

Phase 3 — Response Analysis

API responses were inspected for sensitive data exposure, user information disclosure, and response structure consistency.

Phase 4 — Sequential Identifier Testing

User identifiers were analyzed to determine whether predictable object enumeration was possible.

Phase 5 — Header Inspection

HTTP response headers were inspected for implemented security controls.

Phase 6 — Rate Limiting Review

API rate limiting headers were inspected to evaluate abuse prevention mechanisms.

7. API Security Findings & Analysis

7.1 — Missing Authentication Analysis (JSONPlaceholder)

The /users endpoint returned user-related information without requiring authentication or authorization validation.

Observation

The API allowed unrestricted public access to user-related records.

Security Concern

Unauthenticated access to API endpoints may increase the risk of:

- Information disclosure
- Automated data scraping
- Reconnaissance activities

Severity: High

Representative Response

```
$ curl -I https://jsonplaceholder.typicode.com/users
HTTP/2 200
content-type: application/json; charset=utf-8
x-content-type-options: nosniff
x-ratelimit-limit: 1000
x-ratelimit-remaining: 999
```

A successful HTTP 200 response was returned without authentication requirements.

7.2 — Excessive Data Exposure Analysis (JSONPlaceholder)

Inspection of the /users/1 endpoint response revealed multiple user-related fields returned by default.

Exposed Information

- Full Name
- Email Address
- Phone Number
- Address Information
- Geographic Coordinates
- Company Details

Security Concern

Excessive information exposure may assist attackers during social engineering attacks, reconnaissance activities, and phishing campaigns.

Severity: High

Representative Response

```
$ curl https://jsonplaceholder.typicode.com/users/1 | jq .
{
  "id": 1,
  "name": "Leanne Graham",
```

```
"username": "Bret",
"email": "Sincere@april.biz",
"address": { "street": "Kulas Light", "city": "Gwenborough" },
"phone": "1-770-736-8031 x56442",
"company": { "name": "Romaguera-Crona" }
}
```

7.3 — Sequential Identifier / IDOR Analysis

The API used predictable sequential numerical identifiers.

Observation

User records could be accessed directly using sequential IDs: /users/1, /users/2, /users/3

Security Concern

Predictable identifiers may increase the risk of Insecure Direct Object Reference (IDOR) style attacks if authorization controls are missing.

Severity: **Medium**

Representative Response

```
$ curl https://jsonplaceholder.typicode.com/users/1
$ curl https://jsonplaceholder.typicode.com/users/2
$ curl https://jsonplaceholder.typicode.com/users/3
# Sequential user enumeration observed
```

7.4 — Security Header Analysis (JSONPlaceholder)

HTTP response headers were analyzed to identify implemented security controls.

Headers Observed

- Content-Type
- Cache-Control
- X-Content-Type-Options
- ETag
- x-ratelimit-limit

Positive Security Controls

- HTTPS communication enabled
- Structured JSON responses
- Content-Type protection enabled
- Basic rate limiting identified

Some advanced hardening headers such as Content-Security-Policy were not visibly implemented.

Severity: **Low**

Representative Response

```
$ curl -I https://jsonplaceholder.typicode.com/posts/1
HTTP/2 200
content-type: application/json; charset=utf-8
x-content-type-options: nosniff
```

```
x-ratelimit-limit: 1000
cache-control: max-age=43200
```

7.5 — Rate Limiting Analysis

API responses were inspected for request limiting protections.

Headers Identified

- x-ratelimit-limit
- x-ratelimit-remaining
- x-ratelimit-reset

Security Benefit

- Rate limiting helps reduce: Automated abuse
- Rate limiting helps reduce: Excessive requests
- Rate limiting helps reduce: Scraping activities
- Rate limiting helps reduce: API flooding attempts

Severity: Informational

Representative Response

```
$ curl -I https://jsonplaceholder.typicode.com/photos
HTTP/2 200
x-ratelimit-limit: 1000
x-ratelimit-remaining: 999
x-ratelimit-reset: 1779437649
```

7.6 — ReqRes Public API Analysis

The ReqRes API successfully returned public user-related information through GET requests.

Positive Security Controls

- Proper JSON formatting
- HTTPS communication enabled
- Stable API response structure
- Controlled public response handling

Severity: Informational

Representative Response

```
$ curl https://reqres.in/api/users
{
  "page": 1,
  "per_page": 6,
  "total": 12,
  "data": [
    { "id": 1, "email": "george.bluth@reqres.in",
      "first_name": "George", "last_name": "Bluth" }
  ]
}
```


7.7 — ReqRes Security Header Analysis

HTTP response headers from ReqRes were inspected to identify implemented security protections.

Headers Observed

- Strict-Transport-Security
- X-Frame-Options
- X-Content-Type-Options
- Cache-Control
- Content-Type

Positive Security Controls

- HTTPS enforced
- Security headers implemented
- Proper response handling
- Structured API communication

Severity: **Informational**

Representative Response

```
$ curl -I https://reqres.in/api/users
HTTP/2 200
strict-transport-security: max-age=31536000
x-frame-options: SAMEORIGIN
x-content-type-options: nosniff
content-type: application/json; charset=utf-8
```

8. OWASP API Security Top 10 Mapping

OWASP Category	Related Finding
API1 — Broken Object Level Authorization	Sequential Identifier Enumeration
API2 — Broken Authentication	Missing Authentication Controls
API3 — Broken Object Property Level Authorization	Excessive Data Exposure
API4 — Unrestricted Resource Consumption	Rate Limiting Controls
API8 — Security Misconfiguration	Security Header Configuration

9. Positive Security Controls Identified

HTTPS Communication Enabled

Secure HTTPS communication was enforced during API interactions.

Rate Limiting Mechanisms

Rate limiting related headers were identified during testing.

Structured API Responses

The APIs returned properly formatted and organized JSON responses.

Security Headers Implemented

Security-related headers were identified during API response inspection.

Controlled Public Responses

The APIs returned controlled public data without exposing sensitive internal server information.

10. Remediation Recommendations

Priority	Recommendation	Purpose
High	Implement authentication and authorization mechanisms	Prevent unauthorized access
High	Reduce unnecessary exposure of sensitive user information	Minimize privacy risks
Medium	Replace sequential identifiers with UUIDs	Reduce IDOR-related risks
Medium	Apply field-level authorization validation	Restrict unauthorized data access
Medium	Maintain rate limiting protections	Reduce automated abuse
Medium	Continue implementing security headers	Strengthen API security posture

11. Conclusion

This project successfully demonstrated practical API Security Risk Analysis techniques using publicly available APIs.

The assessment identified several important API security observations including:

- Missing authentication controls
- Excessive data exposure
- Sequential identifier patterns
- Public exposure of user-related information

The project also identified positive security controls including:

- HTTPS communication
- Structured API responses
- Security headers
- Rate limiting protections

This project improved practical understanding of:

- API Security Testing
- Application Security Concepts
- OWASP API Security Risks
- Security Header Analysis
- Risk Assessment
- Security Documentation
- Professional Cybersecurity Reporting

All testing was conducted ethically using read-only testing methodologies.

12. Disclaimer

This project was conducted strictly for educational and internship purposes as part of the Future Interns Cyber Security Internship 2026 program.

All APIs used during testing are publicly available demo/testing APIs intended for learning and educational use.

No exploitation, bypass attempts, unauthorized access, or malicious activities were performed during this assessment.